

Rapport — Étape 3 : parcours de paiement Stripe

Application Fun Kids Animation · Corrections, vérifications et procédure de test · 24 septembre 2026

Élément	Détail
Étape du plan d'action	Étape 3 — Parcours de paiement Stripe (retour client \$5)
Branche / commit	nassim · commit du code : abd01cd
Fichiers modifiés	payments/views.py, payments/urls.py, web/views.py, templates/payments/checkout.html, payments/tests.py
Fichier ajouté	payments/services.py
Base de données	Aucune modification — ni migration, ni changement de structure
Tests automatiques	113 tests : tous OK (100 existants + 13 nouveaux)
Essais réels Stripe	4 scénarios réussis en mode Test (aucun paiement réel)
Résultat global	Étape 3 terminée

1. Objectif

Corriger le parcours de paiement signalé par le client (« le parcours de paiement Stripe bloque lors de la finalisation ») et garantir un statut exact dans tous les cas. Objectifs précis :

- Identifier et supprimer la cause du blocage à la finalisation du paiement.
- Mettre à jour automatiquement la réservation et le paiement après un paiement réussi.
- Transmettre l'e-mail du client à Stripe (reçu de paiement, Stripe Link).
- Traiter correctement un paiement refusé, annulé, abandonné ou en cours de traitement.
- Prévenir le client par e-mail en cas de refus, sans dépendre du webhook.
- Conserver le fonctionnement de « Payer plus tard ».

2. Commits associés

Commit	Date	Message	Contenu
abd01cd	24/09/2026 22:11	fix: finalisation du parcours de paiement Stripe (etape 3)	Correction du blocage, gestion des statuts, synchronisation des refus, Link, 13 tests
85742ac	24/09/2026 22:13	docs: ajout du rapport PDF de l'etape 3	Première version de ce rapport
(suivant)	24/09/2026	docs: mise a jour du rapport de l'etape 3	Version actuelle : procédure pas à pas, limites

Commit de référence pour le code : abd01cd. Détail : `git show abd01cd --stat`. Étapes précédentes : e8ed796 (sécurité), c99617d (comptes).

3. Récapitulatif des corrections

#	Correction	Demande client (\$5)	Statut
1	Blocage à la finalisation supprimé (incompatibilité avec le SDK Stripe 15)	Corriger le parcours jusqu'à la confirmation	✓ Réalisé
2	Réservation confirmée et paiement « Réussi » automatiquement après paiement	Mise à jour automatique du statut	✓ Réalisé

#	Correction	Demande client (§5)	Statut
3	E-mail du client transmis à Stripe (reçu + Link) ; association paiement / réservation / compte	E-mail client, association, Link	✓ Réalisé
4	Paiement refusé : statut « Échoué », e-mail au client, nouvel essai possible	Tester le paiement échoué	✓ Réalisé
5	Paiement annulé ou abandonné : réservation laissée en attente	Tester le paiement annulé	✓ Réalisé
6	Paiement « en cours de traitement » laissé en attente (plus marqué échoué)	Statut exact	✓ Réalisé
7	« Payer plus tard » inchangé	Déjà fonctionnel	✓ Réalisé
8	Affichage du formulaire Stripe dans un vrai navigateur (Link, message de refus)	Vérification visuelle	À tester au navigateur

4. Description détaillée

4.1 Cause du blocage à la finalisation

Avant

Le projet utilise la version 15 du SDK Stripe, dont les objets ne sont plus des dictionnaires Python : la méthode `get()` n'existe plus. Au retour de Stripe, la page de succès appelait `intent.get(...)`, ce qui levait une erreur interceptée silencieusement : le client voyait « Le paiement n'a pas pu être vérifié » et la réservation restait « En attente » alors que Stripe avait bien encaissé. Le même défaut cassait le second essai de paiement (erreur 500) et le webhook. Les tests existants ne le détectaient pas car ils simulaient Stripe avec de simples dictionnaires. **Constaté en conditions réelles** : paiement « succeeded » chez Stripe, réservation restée « PENDING », journal : « Vérification du paiement impossible auprès de Stripe : get ».

Après

Les objets Stripe sont convertis en dictionnaires avant toute lecture (`as_dict`), sur la page de succès, la création/réutilisation du paiement et le webhook. Trois tests utilisent désormais de vrais objets du SDK pour empêcher tout retour de ce défaut.

Fichier : `payments/services.py`, `payments/views.py`, `web/views.py`

4.2 Traitement unique de l'état du paiement

Avant

Chaque point d'entrée (page de succès, webhook) avait sa propre logique. Tout statut autre que « réussi » était enregistré comme « Échoué », y compris un paiement encore en cours de traitement par la banque.

Après

Nouveau module `payments/services.py` : l'état réel du `PaymentIntent` est répercuté de façon identique partout. Réussi → paiement « Réussi » + réservation « Confirmée » (e-mails de confirmation). En cours → en attente. Refusé → « Échoué » + e-mail au client (une seule fois). Annulé → « Échoué », réservation en attente. Abandonné → en attente. Un paiement déjà encaissé n'est jamais rétrogradé.

Fichier : `payments/services.py`

4.3 Carte refusée

Avant

Un refus de carte n'entraîne pas de redirection : seul le navigateur en était informé. Sans webhook actif (cas du local), rien n'était enregistré et aucun e-mail n'était envoyé.

Après

Le formulaire appelle `/api/payments/sync-status/` après un refus. Le serveur lit l'état chez Stripe (jamais une valeur envoyée par le navigateur), enregistre l'échec et prévient le client. Le point d'accès est réservé au titulaire de la réservation. Le client peut réessayer avec une autre carte sur la même réservation.

Fichier : `payments/views.py`, `templates/payments/checkout.html`

4.4 E-mail du client et Stripe Link

Avant

Le formulaire de carte ne connaissait pas l'adresse du client : Stripe Link la redemandait. L'interface Stripe s'affichait dans la langue du navigateur.

Après

L'e-mail et le nom du compte sont transmis au formulaire Stripe (Link reconnaît le client), l'interface est forcée en français. L'e-mail reste aussi transmis côté serveur (`receipt_email` et métadonnées).

Fichier : `templates/payments/checkout.html`

5. Vérifications effectuées

5.1 Tests automatiques

Groupe de tests	Ce qui est vérifié	Résultat
Statuts (6)	Réussi, en cours, refusé (e-mail unique), annulé, abandonné, pas de rétrogradation	✓ Réalisé
Page de succès (1)	Message « en cours de traitement » pour un paiement processing	✓ Réalisé
Synchronisation (2)	Refus enregistré ; accès refusé à un autre client et à un visiteur anonyme	✓ Réalisé
Formulaire (1)	E-mail du client transmis au formulaire Stripe	✓ Réalisé
SDK Stripe réel (3)	Page de succès, réutilisation du paiement, webhook signé avec de vrais objets Stripe	✓ Réalisé
Suite complète	113 tests, dont les 100 précédents (étapes 1 et 2)	✓ Réalisé

5.2 Essais réels avec Stripe (mode Test)

Serveur de vérification lancé en local sur **`http://127.0.0.1:8011`**, compte Client 2 connecté, paiements effectués avec les moyens de paiement de test officiels de Stripe (aucune carte réelle). Les réservations créées pour l'essai ont été supprimées ensuite.

Scénario	Résultat obtenu	Statut
Page de paiement	HTTP 200, e-mail du client présent dans le formulaire, interface en français	✓ Réalisé
Création du paiement	<code>receipt_email</code> et <code>metadata.client_email</code> = <code>client2@funkidz.fr</code>	✓ Réalisé
1. Paiement réussi (carte 4242...)	Stripe « succeeded » → réservation CONFIRMED, paiement SUCCEEDED	✓ Réalisé
2. Carte refusée (4000 0000 0000 0002)	Stripe « Your card was declined » → paiement FAILED, réservation PENDING, e-mail d'échec envoyé	✓ Réalisé

Scénario	Résultat obtenu	Statut
2 bis. Nouvel essai avec une carte valide	Même paiement Stripe réutilisé → réservation CONFIRMED, paiement SUCCEEDED	✓ Réalisé
3. Abandon / annulation	Page d'annulation 200 → réservation PENDING, paiement PENDING	✓ Réalisé
4. Payer plus tard	Réservation PENDING, aucun paiement créé	✓ Réalisé
Erreurs serveur	Aucune (0 traceback dans le journal)	✓ Réalisé

5.3 E-mails émis pendant les essais (journal du serveur)

E-mail	Destinataire(s)
Confirmation de votre réservation Funkidz #... — C'est la fête !	client2@funkidz.fr
PAIEMENT CONFIRMÉ #... — 150.00€	admin@funkidz.fr, nassim2389@hotmail.com, contact@funkidz.fr
Information concernant le règlement de votre réservation Funkidz #... (échec)	client2@funkidz.fr

En local, les e-mails sont affichés dans la console du serveur : leur délivrance réelle relève de l'étape 4 (Brevo).

6. Résultats obtenus

Sortie de la suite de tests complète sur le commit final :

```
$ venv/bin/python manage.py test
Found 113 test(s).
System check identified no issues (0 silenced).
Ran 113 tests in 118.599s

OK
```

Sortie du script d'essai réel (extrait) :

```
[1] Succès | create-intent 200 | receipt_email = client2@funkidz.fr
Stripe=succeeded | page 200 | réservation=CONFIRMED | paiement=SUCCEEDED
[2] Refus | Stripe répond : Your card was declined.
sync-status 200 {'outcome': 'failed', 'payment_status': 'FAILED', 'booking_status':
'PENDING'}
e-mail échec envoyé=True
Nouvel essai carte valide (même intent réutilisé=True) | réservation=CONFIRMED |
paiement=SUCCEEDED
[3] Abandon | page annulation 200 | réservation=PENDING | paiement=PENDING
[4] Payer plus tard | réservation=PENDING | paiements=0
```

Objectif	Résultat obtenu	Statut
Plus de blocage à la finalisation	Paiement réussi → réservation confirmée automatiquement	✓ Réalisé
E-mail client transmis à Stripe	Reçu, métadonnées et formulaire (Link) renseignés	✓ Réalisé
Refus correctement traité	Statut « Échoué », e-mail au client, nouvel essai possible	✓ Réalisé
Annulation / abandon	Réservation laissée en attente de paiement	✓ Réalisé
Paiement en cours	Reste en attente, message adapté	✓ Réalisé
Aucune régression	113/113 tests OK, « Payer plus tard » inchangé	✓ Réalisé

Conclusion : l'étape 3 est terminée et stable côté serveur. Le blocage signalé par le client venait d'une incompatibilité avec la version du SDK Stripe ; il est corrigé et vérifié avec de vrais paiements de test. Seule

reste la vérification visuelle du formulaire dans un navigateur (section 7.3).

7. Comment tester soi-même

7.1 Tests automatiques

```
venv/bin/python manage.py test payments
```

7.2 Lancer le serveur (port 8010)

```
venv/bin/python manage.py runserver 0.0.0.0:8010
```

7.3 Procédure pas à pas dans le navigateur

1. Sur le serveur, depuis la racine du projet, lancer : `venv/bin/python manage.py runserver 0.0.0.0:8010`
2. Depuis un poste du réseau, ouvrir **<http://192.168.1.100:8010/>** (ou **<http://127.0.0.1:8010/>** sur le serveur lui-même).
3. Se connecter avec le compte de test **client2@funkidz.fr** (mot de passe communiqué séparément).
4. Créer une réservation depuis « Réserver » puis cliquer sur « Payer par carte ».
5. Vérifier que l'adresse e-mail du compte est déjà renseignée dans le formulaire Stripe (Link) et que l'interface est en français.
6. Saisir la carte **4242 4242 4242 4242**, date 12/34, CVC 123, puis valider : la page « Paiement confirmé » s'affiche et la réservation passe à « Confirmée » dans « Mon espace ».
7. Créer une seconde réservation et payer avec **4000 0000 0000 0002** : un message de refus en français s'affiche sous le formulaire ; dans l'administration (/admin/), le paiement est « Échoué » et la réservation « En attente ».
8. Sur cette même réservation, cliquer de nouveau sur « Payer » et utiliser 4242 4242 4242 4242 : la réservation passe à « Confirmée ».
9. Créer une troisième réservation, ouvrir la page de paiement puis la quitter sans payer : la réservation reste « En attente de paiement ».
10. Contrôler dans le tableau de bord Stripe (mode Test) que les paiements apparaissent avec l'adresse du client.

Cartes de test Stripe utiles :

Scénario	Carte de test Stripe	Résultat attendu
Paiement réussi	4242 4242 4242 4242	Page « Paiement confirmé », réservation « Confirmée » dans Mon espace, e-mail de confirmation
Carte refusée	4000 0000 0000 0002	Message de refus en français sous le formulaire, paiement « Échoué » dans l'admin, e-mail d'échec
Fonds insuffisants	4000 0000 0000 9995	Même comportement que la carte refusée
Authentification 3D Secure	4000 0027 6000 3184	Fenêtre de validation Stripe, puis paiement confirmé
Abandon	—	Quitter la page : la réservation reste « En attente de paiement » et peut être payée plus tard
Stripe Link	—	L'adresse e-mail du compte est déjà renseignée dans le formulaire

Pour toutes les cartes : date d'expiration future (ex. 12/34), CVC à 3 chiffres au choix. Les paiements apparaissent dans le tableau de bord Stripe, en mode Test.

8. Points d'attention

- Base de données : aucune modification. Rien à signaler pour le dictionnaire de données.
- Le webhook reste facultatif en local ; en production, renseigner STRIPE_WEBHOOK_SECRET (voir étape 1).
- Les e-mails ne sont réellement délivrés qu'une fois Brevo configuré (étape 4).
- Limite de la vérification : le formulaire de carte (affichage, Link, message de refus, 3D Secure) n'a pas pu être contrôlé dans un navigateur graphique depuis le serveur ; le code et les réponses du serveur sont vérifiés, l'affichage reste à contrôler une fois avec la section 7.3.
- Limite de la vérification : les liens de paiement générés depuis l'administration (Stripe Checkout) bénéficient de la même correction mais n'ont pas été payés en conditions réelles. Le parcours client (réservation puis « Payer par carte ») est, lui, entièrement vérifié.

9. Suite du plan d'action

Étape	Contenu	Statut
1	Failles de sécurité critiques (commit e8ed796)	✓ Réalisé
2	Comptes et connexion multi-clients (commit c99617d)	✓ Réalisé
3	Parcours de paiement Stripe (commit abd01cd)	✓ Réalisé
4	Brevo et e-mails de bout en bout	À faire
5 à 10	Créneaux, administration, calendrier, livraison du 29/09, soutenance	À faire